

Pulse — מעבר ל-MySQL + Express + NodeJS

תיעוד טכני: מה בוצע בשלב זה, אילו טכנולוגיות, ואיך מריצים

פרויקט: Pulse (לקוח React + שרת REST) · בסיס נתונים: MySQL 8 דרך Docker · שרת: Express על פורט 4000

1. סקירה כללית — מה נעשה בשלב זה

בפרק ה' הלקוח עבד מול `json-server` — שרת זמני שקורא מקובץ `db.json`, ללא בסיס נתונים אמיתי ועם אימות חלקי. בפרק ו' **החלפנו את כל שכבת השרת** בשרת אמיתי שכתוב ב-NodeJS עם **Express**, המחובר לבסיס נתונים **MySQL** הרץ בתוך **Docker**. לקוח ה-React כמעט לא השתנה — הוא ממשיך לעבוד מול אותו חוזה REST, רק שמאחוריו עכשיו שרת ובסיס נתונים אמיתיים.

היבט	לפני (פרק ה')	אחרי (פרק ו')
שרת	json-server מעל db.json	NodeJS + Express אמיתי
אחסון	קובץ JSON	MySQL 8 (בטבלאות, עם קשרים)
אימות	השוואת סיסמה בצד הלקוח	אימות בצד השרת, סיסמאות מוצפנות
מחיקה	מחיקה פיזית	מחיקה רכה (soft delete)
מזהים	מספור רץ	UUID ייחודי

2. ארכיטקטורה

הזרימה בנויה בשכבות ברורות, כל אחת עם אחריות יחידה:

React (לקוח) —HTTP—> Express Routes —> Queries שכבת —> MySQL (ב-Docker)
:5173 :4000 (פונקציות ייעודיות) :3307

- **Routes** — נתיבי ה-REST (דקים): מקבלים בקשה, מאמתים הרשאות, וקוראים לשכבת ה-Queries.
- **Queries** — "הפונקציות הייעודיות" שההנחיה דורשת: כל פעולה על בסיס הנתונים (קריאה/הוספה/עדכון/מחיקה) ממושת בפונקציה ייעודית עם שאילתת SQL מאובטחת.
- **Middleware** — זיהוי המשתמש המחובר (טוקן), אכיפת בעלות, וטיפול מרוכז בשגיאות.
- **db.js** — מאגר חיבורים (connection pool) ל-MYSQL.

מבנה תיקיות השרת:

```
server/  
├─ docker-compose.yml      הגדרת MySQL ב-Docker  
├─ .env                    DB-פורטים + פרטי חיבור ל  
├─ sql/schema.sql          הגדרת כל הטבלאות  
├─ scripts/seed.js         (כולל הצפנת סיסמאות) db.json מ-DB-מילוי ה  
└─ src/  
    ├─ index.js db.js app.js  
    ├─ middleware/         (currentUser, ownership, errorHandler)  
    ├─ queries/            (users, todos, posts, comments, albums, photos)  
    └─ routes/             (auth, users, todos, posts, comments, albums, photos, admin)
```

3. הטכנולוגיות שבהן השתמשנו

טכנולוגיה	תפקיד
Node.js	סביבת הריצה של השרת
Express	מסגרת השרת – הגדרת נתיבי ה-REST API
MySQL 8	בסיס הנתונים היחסי (טבלאות, מפתחות זרים, אינדקסים)
Docker / docker-compose	הרצת MySQL בקונטיינר מבודד, ללא התקנה ידנית
mysql2	הדרייבר שמחבר את Node ל-MySQL; תומך ב-prepared statements
bcryptjs	הצפנת (hashing) סיסמאות
cors	מאפשר ללקוח (פורט 5173) לפנות לשרת (פורט 4000)
dotenv	טעינת הגדרות מקובץ <code>.env</code>
React + Vite	לקוח ה-Web (ללא שינוי מהותי)
Postman	אוסף בדיקות לכל נתיבי ה-API

4. איך מימשנו כל דרישה

4.1 בסיס הנתונים MySQL דרך Docker

במקום להתקין MySQL ידנית, הגדרנו אותו בקובץ `docker-compose.yml`. הפקודה `npm run db:up` (שמריצה `docker compose up -d`) מקימה קונטיינר MySQL 8. קובץ הסכמה `sql/schema.sql` מחובר לתיקיית האתחול של הקונטיינר (`/docker-entrypoint-initdb.d`), כך שכל הטבלאות נוצרות אוטומטית בעליית הקונטיינר. היתרון: סביבה זהה לכל חברי הצוות, ניתנת לאיפוס בפקודה אחת, ללא התקנות מסובכות.

4.2 ארכיטקטורת הטבלאות וטבלת הסיסמאות הנפרדת

בנינו טבלאות עבור `users, todos, posts, comments, albums, photos` עם מפתחות זרים (FK) ואינדקסים. הסיסמאות **אינן** נשמרות בטבלת המשתמשים אלא בטבלה נפרדת ומוגנת בשם `credentials` – אין לה אף נתיב API, והיא נקראת אך ורק בתהליך ההתחברות. הסיסמאות שמורות כ-`hash` (bcrypt), כך שגם צפייה ישירה בבסיס הנתונים לא חושפת אותן.

4.3 שרת Express ונתיבי REST

בנינו מסגרת Express עם נתיבים בסגנון `jsonplaceholder`, התומכים בכל הפעולות: `GET, POST, PUT, PATCH, DELETE` לכל משאב, כולל פרמטרים בכתובת לסינון, מיון ועימוד (`?userId= &_sort= &_page= &_per_page=`). כל שאילתה משתמשת ב-`prepared statements` (סימני `?`) – הגנה מובנית מפני SQL Injection.

4.4 אימות בצד השרת (לא בלקוח)

ההתחברות מתבצעת בנתיב `POST /login`: השרת מאמת את הסיסמה מול טבלת `credentials` ומחזיר את פרטי המשתמש (ללא סיסמה) יחד עם **טוקן**. הלקוח שומר את הטוקן ב-Local Storage ושולח אותו בכותרת `Authorization` בכל בקשה. כך הסיסמה אף פעם לא מוחזרת ולא נשמרת בצד הלקוח.

4.5 בעלות (Ownership) ומחיקה רכה

- **בעלות בצד השרת:** עדכון/מחיקה של פוסט או תגובה מתאפשרים רק אם הם שייכים למשתמש המחובר – נאכף ב-middleware בשרת (מחזיר 403 לזר). זאת משום שאת הלקוח אפשר לעקוף.
- **מחיקה רכה:** מחיקה אינה מוחקת פיזית אלא מסמנת `is_deleted = 1` והרשומה מסוננת משאילתות הקריאה. זו התשובה לשאלה "מה זו מחיקה?" שבהנחיות – המידע נשמר וניתן לשחזור.

4.6 שיפורים נוספים (יעילות ואבטחה)

שיפור	כיצד
מזהים ייחודיים (UUID)	כל הטבלאות משתמשות ב-UUID במקום מספור רץ – לא ניתן לנחש/לספור רשומות.
כתובת URL אינפורמטיבית	ב-URL מופיע ה-username (למשל <code>/users/maya_c/todos</code>) – קריא וייחודי; ה-UUID נשאר פנימי.
מחיקה בקריאת רשת אחת	מחיקת פוסט/אלבום היא קריאה אחת; השרת מוחק את הנגזרות (comments/photos) ב-cascade.
חשיפת מינימום	<code>GET /users</code> מחזיר רק <code>id, name, username</code> ; פרטים מלאים רק למשתמש עצמו או למנהל.
ולידציה בשרת	אורך סיסמה ופורמט שם משתמש נאכפים גם בשרת, לא רק בלקוח.
עימוד תמונות	מוצגות 5 תמונות, וכפתור "טען עוד" מביא 5 נוספות בכל לחיצה – קריאה אחת לעמוד, ללא כפילויות.

5. שאלות אפשריות מהמרצה – ותשובות

ש: למה הרצתם את MySQL דרך Docker ולא התקנתם אותו?

ת: כדי לקבל סביבה זזה, נקייה וניתנת לשחזור לכל חברי הצוות, ללא התקנה והגדרה ידנית של MySQL. הקונפיגורציה כולה בקובץ אחד (`docker-compose.yml`), והסכמה נטענת אוטומטית בעליית הקונטיינר.

ש: איפה נשמרות הסיסמאות, ואיך הן מוגנות?

ת: בטבלה נפרדת `credentials` (לא בטבלת המשתמשים), כ-hash של bcrypt. לטבלה אין אף נתיב API והיא נקראת רק בתהליך ה-login. הסיסמה לעולם לא מוחזרת בתשובת השרת.

ש: איך אתם מונעים SQL Injection?

ת: כל השאילתות משתמשות ב-prepared statements (פרמטרים עם `?` דרך mysql2), כך שקלט המשתמש לעולם אינו משורשר ישירות לתוך ה-SQL. בנוסף, שדות מיון/סינון מוגבלים לרשימה לבנה (whitelist).

ש: "מה זו מחיקה?" (הנקודה למחשבה בהנחיות)

ת: בחרנו במחיקה רכה – סימון `is_deleted=1` במקום מחיקה פיזית. כך המידע נשמר, ניתן לשחזור, ומחיקת רשומת-אב מסמנת גם את הנגזרות שלה (פוסט→תגובות, אלבום→תמונות).

ש: מה ההגנה אם מישהו עוקף את הלקוח?

ת: כל הגנה שקיימת בלקוח קיימת גם בשרת: אימות, בדיקת ייחודיות שם משתמש, ולידציה, ובדיקות בעלות. השרת לעולם אינו סומך על הלקוח.

ש: למה UUID ולא מספור רץ? ולמה username בכתובת?

ת: מספור רץ מאפשר לנחש ולספור רשומות (לדוגמה לדפדף בין UUID). (`/users/1`, `/users/2`) מונע זאת. בכתובת ה-URL השתמשנו ב-username כדי שתהיה גם קריאה/אינפורמטיבית וגם לא חושפת מזהה רץ.

ש: איך נמנעות קריאות רשת מיותרות/כפולות?

ת: כל פעולה שולחת רק את מה שצריך – מחיקה היא קריאה אחת (cascade בשרת), והעימוד מביא רק את העמוד הבא. בנוסף הלקוח שומר תוצאות ב-cache מקומי כדי לא לחזור ולשאול את השרת.

ש: מהן "פונקציות הייעודיות" שביקשה ההנחיה?

ת: שכבת `queries/` – לכל משאב מודול עם פונקציות ייעודיות (list, getByld, create, update, softDelete), ה-`routes` דקים וקוראים להן. בדקנו אותן בסקריפט בדיקה ייעודי. (ownerOf

6. איך מריצים

דרישות מוקדמות: Node.js 18 + ו-Docker Desktop (צריך להיות פתוח ופעיל).

1 מפעילים את **Docker Desktop** ומחכים שיהיה פעיל.

2 בטרמינל — מתקינים ומריצים את השרת + בסיס הנתונים:

```
cd server
npm install           # פעם ראשונה בלבד
npm run db:up         # Docker-ב MySQL מפעיל את
npm run seed          # (כולל הצפנת סיסמאות) db.json-מ DB-ממלא את ה
npm start             # http://localhost:4000 מריץ את השרת על
```

3 בטרמינל שני — מריצים את הלקוח:

```
cd client
npm install           # פעם ראשונה בלבד
npm run dev           # http://localhost:5173
```

4 כנסיים עם משתמש בדיקה (טבלה למטה).

פקודות שימושיות (בתוך **server**):

מה עושה	פקודה
הפעלה / כיבוי של בסיס הנתונים	<code>npm run db:up / db:down</code>
איפוס מלא והקמת DB נקי	<code>npm run db:reset</code>
מילוי מחדש של הנתונים	<code>npm run seed</code>
בדיקת שכבת בסיס הנתונים	<code>npm run test:queries</code>

משתמשי בדיקה

שם משתמש	סיסמה	תפקיד
shaked.h	pulse123	מנהל (admin)
maya_c	hello456	משתמש רגיל
dan_l	sunny789	משתמש רגיל

הערה: בפעמים הבאות מספיק `npm run db:up` ואז `npm start` (אין צורך ב-install/seed שוב).